

Guía paso a paso: Escalar el proyecto a Cucumber y API REST (Spring Boot)

Proyecto base: `user-quality-demo` (gestión de usuarios + pruebas unitarias, integración y aceptación).

Índice

1. Extender a API REST con Spring Boot
 - 1.1. Actualizar `pom.xml`
 - 1.2. Crear clase principal `UserQualityDemoApplication`
 - 1.3. Crear controlador REST `UserController`
 - 1.4. Probar la API (manual y con tests)
2. Añadir Cucumber para pruebas de aceptación BDD
 - 2.1. Actualizar `pom.xml` con dependencias de Cucumber
 - 2.2. Crear archivo `.feature` con escenarios Given-When-Then
 - 2.3. Crear clase runner de Cucumber
 - 2.4. Crear step definitions conectadas al `UserService`
 - 2.5. Ejecutar las pruebas de Cucumber

1. Extender a API REST con Spring Boot

1.1. Actualizar `pom.xml`

Sustituye el contenido actual por una versión con Spring Boot (o ajusta lo necesario):

```

<<<xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>user-quality-demo</artifactId>
  <version>1.1.0</version>
  <packaging>jar</packaging>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.5</version>
  </parent>

  <properties>
    <java.version>17</java.version>
  </properties>

```

```
<dependencies>
  <!-- Web / REST -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- JUnit 5 ya viene integrado con Spring Boot -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>

  <!-- Mockito explícito (opcional, ya incluido en starter-test) -->
  <dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-core</artifactId>
    <version>5.11.0</version>
    <scope>test</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
</project>
```

1.2. Clase principal `UserQualityDemoApplication`

Crea:

```
// src/main/java/com/example/usermanagement/UserQualityDemoApplication.java
package com.example.usermanagement;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

/**
 * Punto de entrada de la aplicación Spring Boot.
 */
@SpringBootApplication
public class UserQualityDemoApplication {
```

```
    public static void main(String[] args) {  
        SpringApplication.run(UserQualityDemoApplication.class, args);  
    }  
}
```

1.3. Controlador REST `UserController`

Usaremos el `UserService` y el `InMemoryUserRepository` como backend simple.

```
// src/main/java/com/example/usermanagement/api/UserController.java  
package com.example.usermanagement.api;  
  
import com.example.usermanagement.model.User;  
import com.example.usermanagement.repository.InMemoryUserRepository;  
import com.example.usermanagement.repository.UserRepository;  
import com.example.usermanagement.service.UserService;  
import org.springframework.http.ResponseEntity;  
import org.springframework.web.bind.annotation.*;  
  
import java.util.List;  
  
/**  
 * Controlador REST para gestionar usuarios.  
 * Este es el "punto de contacto" con el mundo exterior.  
 */  
@RestController  
@RequestMapping("/api/users")  
public class UserController {  
  
    private final UserService userService;  
  
    // Para simplificar, instanciamos el repositorio en memoria aquí.  
    public UserController() {  
        UserRepository repo = new InMemoryUserRepository();  
        this.userService = new UserService(repo);  
    }  
  
    @PostMapping  
    public ResponseEntity<User> create(@RequestBody User user) {  
        User created = userService.createUser(user.getName(), user.getEmail());  
        return ResponseEntity.ok(created);  
    }  
  
    @GetMapping  
    public ResponseEntity<List<User>> getAll() {  
        return ResponseEntity.ok(userService.getAllUsers());  
    }  
  
    @PutMapping("/{id}")  
    public ResponseEntity<User> update(@PathVariable Long id,
```

```
        @RequestBody User user) {
            User updated = userService.updateUser(id, user.getName(),
user.getEmail());
            return ResponseEntity.ok(updated);
        }

        @DeleteMapping("/{id}")
        public ResponseEntity<Void> delete(@PathVariable Long id) {
            userService.deleteUser(id);
            return ResponseEntity.noContent().build();
        }
    }
}
```

Para producción, lo ideal sería usar inyección de dependencias con `@Bean` o `@Service` y un repositorio real (JPA, etc.). Aquí mantenemos la simplicidad didáctica.

1.4. Probar la API REST

1. Ejecutar la aplicación:

```
mvn spring-boot:run
```

2. Probar con `curl` o Postman:

- Crear usuario:

```
curl -X POST http://localhost:8080/api/users \
-H "Content-Type: application/json" \
-d '{"name":"Cesar","email":"cesar@example.com"}'
```

- Listar usuarios:

```
curl http://localhost:8080/api/users
```

- Actualizar usuario:

```
curl -X PUT http://localhost:8080/api/users/1 \
-H "Content-Type: application/json" \
-d '{"name":"Cesar Actualizado","email":"cesar@example.com"}'
```

- Eliminar usuario:

```
curl -X DELETE http://localhost:8080/api/users/1
```

En clase puedes conectar esto con las pruebas de integración: ahora el servicio se expone como API y se puede probar end-to-end.

2. Añadir Cucumber para pruebas de aceptación BDD

Ahora vamos a reemplazar/acompañar las pruebas de aceptación JUnit por **escenarios Cucumber**.

2.1. Actualizar `pom.xml` con dependencias de Cucumber

Añade estas dependencias dentro de `<dependencies>`:

```
<!-- Cucumber JUnit Platform -->
<dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-java</artifactId>
  <version>7.15.0</version>
  <scope>test</scope>
</dependency>

<dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-junit-platform-engine</artifactId>
  <version>7.15.0</version>
  <scope>test</scope>
</dependency>
```

2.2. Crear archivo `.feature` con escenarios Given–When–Then

Crea el directorio de recursos de test:

```
src/test/resources/features/user_management.feature
```

Contenido:

Feature: Gestión de usuarios

Como administrador
Quiero evitar emails duplicados
Para garantizar la unicidad de los usuarios

Scenario: Rechazar creación de usuario con email ya existente

Given existe un usuario con email "existing@example.com"

```
When intento crear un usuario con nombre "Nuevo" y email
"existing@example.com"
Then el sistema debe rechazar la creación con el mensaje "El email ya está
registrado"

Scenario: Rechazar creación de usuario sin nombre
Given no existe usuario con email "noname@example.com"
When intento crear un usuario con nombre "" y email "noname@example.com"
Then el sistema debe rechazar la creación con el mensaje "El nombre es
obligatorio"
```

2.3. Crear clase runner de Cucumber

```
// src/test/java/com/example/usermanagement/cucumber/CucumberTest.java
package com.example.usermanagement.cucumber;

import io.cucumber.junit.platform.engine.Cucumber;

/**
 * Runner de Cucumber usando JUnit Platform.
 * Al ejecutar mvn test, Cucumber buscará los .feature y steps.
 */
@Cucumber
public class CucumberTest {
    // No necesita código adicional.
}
```

2.4. Crear step definitions conectadas al UserService

```
// src/test/java/com/example/usermanagement/cucumber/UserSteps.java
package com.example.usermanagement.cucumber;

import com.example.usermanagement.repository.InMemoryUserRepository;
import com.example.usermanagement.repository.UserRepository;
import com.example.usermanagement.service.UserService;
import io.cucumber.java.Before;
import io.cucumber.java.en.*;

import static org.junit.jupiter.api.Assertions.*;

public class UserSteps {

    private UserRepository repo;
    private UserService service;
    private Exception capturedException;

    @Before
```

```
public void setup() {
    repo = new InMemoryUserRepository();
    service = new UserService(repo);
    capturedException = null;
}

@Given("existe un usuario con email {string}")
public void existe_un_usuario_con_email(String email) {
    service.createUser("Existente", email);
}

@Given("no existe usuario con email {string}")
public void no_existe_usuario_con_email(String email) {
    // No hacemos nada: el repositorio está vacío
}

@When("intento crear un usuario con nombre {string} y email {string}")
public void intento_crear_un_usuario_con_nombre_y_email(String name, String
email) {
    try {
        service.createUser(name, email);
    } catch (Exception e) {
        capturedException = e;
    }
}

@Then("el sistema debe rechazar la creación con el mensaje {string}")
public void el_sistema_debe_rechazar_la_creacion_con_el_mensaje(String
expectedMessage) {
    assertNotNull(capturedException, "Se esperaba una excepción");
    assertEquals(expectedMessage, capturedException.getMessage());
}
}
```

Aquí se ve claramente el puente entre **escenarios de negocio** y **lógica de servicio**. Es un excelente punto para discutir trazabilidad entre requisitos y pruebas.

2.5. Ejecutar las pruebas de Cucumber

Simplemente:

```
mvn test
```

- JUnit ejecutará `CucumberTest`.
- Cucumber leerá `user_management.feature`.
- Los steps en `UserSteps` se ejecutarán contra el `UserService` real.

En clase puedes mostrar:

- El **.feature** como especificación legible por negocio.
 - Los steps como “implementación técnica” de esa especificación.
 - Cómo los mensajes de error del servicio se convierten en criterios de aceptación.
-

Conclusiones

- **Spring Boot** lleva el mismo servicio a un contexto más cercano a producción: API REST, endpoints, HTTP.
- **Cucumber** convierte las pruebas de aceptación en artefactos BDD, alineados con historias de usuario.

Con este archivo **.md** puedes:

- Entregarlo como guía de laboratorio.
- Usarlo como base para una práctica donde los estudiantes:
 - Extienden nuevos endpoints.
 - Añaden nuevos escenarios Cucumber para otras reglas de negocio.